Implementación del monitor de signos vitales

Parte 3: comunicación web local y fundamentos de IoT con ESP32-S3

Frecuencia cardíaca, SpO2, temperatura y visualización desde un celular o computador

Profesor: Juan Diego Lemos

Descripción breve de la práctica. En esta tercera parte del proyecto, el monitor de signos vitales se convertirá en un pequeño dispositivo de Internet de las Cosas. El ESP32-S3 actuará como punto de acceso Wi-Fi, creará una red local propia y ejecutará un servidor web. Desde un celular o computador conectado a esa red, los estudiantes podrán visualizar las variables fisiológicas adquiridas por el sistema: frecuencia cardíaca, saturación de oxígeno y temperatura.

Mensaje inicial. Hasta este punto ya se han integrado sensores, acondicionamiento analógico, conversión ADC, comunicación I2C y visualización local. Esta práctica agrega una capa muy importante: comunicación en red. Es un paso significativo porque acerca el proyecto al tipo de sistemas biomédicos conectados que se desarrollan actualmente en la industria: dispositivos que miden, procesan, comunican y muestran información a otros equipos.

Resultados de aprendizaje

- Relacionar las capas del stack TCP/IP con una aplicación real implementada en un ESP32-S3.
- Identificar la función de la dirección MAC en la capa de acceso a la red.
- Crear un punto de acceso Wi-Fi local con el ESP32-S3 y consultar su dirección IP.
- Implementar un servidor web básico sobre el puerto 80 usando el microcontrolador.
- Distinguir entre una página web estática y una página dinámica que consulta datos periódicamente.
- Utilizar JavaScript básico para actualizar información en una interfaz web.
- Integrar adquisición biomédica, procesamiento, visualización local y comunicación web.
- Reconocer la utilidad de una interfaz hombre-máquina basada en navegador para sistemas embebidos biomédicos.

Materiales requeridos

- Tarjeta ESP32-S3 configurada en Arduino IDE.
- Pantalla OLED SSD1306 conectada por I2C.
- Sensor MAX30100 para frecuencia cardíaca y SpO2.
- Circuito de acondicionamiento del termistor NTC funcionando y calibrado.
- Sensor termistor NTC 3950 de 10 kΩ tipo sonda.
- Celular o computador con Wi-Fi y navegador web.
- Cable USB para programación y monitor serial.
- Arduino IDE con las bibliotecas necesarias instaladas: WiFi, WebServer, Wire, Adafruit GFX, Adafruit SSD1306 y MAX30100lib.

1. De un monitor local a un dispositivo IoT

Internet de las Cosas (IoT) es el concepto de conectar objetos físicos a redes de comunicación para que puedan enviar, recibir, procesar o visualizar información. En esta práctica, el objeto conectado será el monitor de signos vitales desarrollado en las fases anteriores. La idea no es depender de internet externo, sino crear una red local con el propio ESP32-S3.

Diferencia importante. Cuando el ESP32 funciona como hotspot, el celular se conecta a la red creada por el microcontrolador. Esa red permite comunicación local entre el celular y el ESP32, pero no necesariamente da acceso a internet. El celular puede mostrar un aviso de "sin internet"; eso es normal. Lo que necesitamos es comunicación local para consultar la página del monitor.

2. Relación con el stack TCP/IP

Capa TCP/IP	En la práctica	Qué observará el estudiante
Acceso a la red	Wi-Fi creado por el ESP32-S3	SSID, dirección MAC, conexión del celular a la red local
Internet / IP	Dirección IP del ESP32 y del cliente	ESP32 como 192.168.4.1 y celular como cliente de esa red
Transporte	TCP hacia el puerto 80	El navegador abre una conexión TCP con el servidor web
Aplicación	HTTP, HTML, JSON y JavaScript	Página web que muestra o actualiza los signos vitales

3. Fase 1: Wi-Fi, acceso a la red y dirección MAC

Wi-Fi es una tecnología de acceso a la red. En el modelo TCP/IP se ubica en la capa inferior, encargada de permitir que dos dispositivos se conecten físicamente por radiofrecuencia. En esta práctica, el ESP32-S3 usará su interfaz Wi-Fi para crear una red local.

La dirección MAC es un identificador asociado a la interfaz de red. Es como una "placa" de la tarjeta Wi-Fi. A diferencia de la dirección IP, que puede cambiar según la red, la MAC identifica la interfaz de comunicación a nivel de acceso a la red.

Código 1. Obtener la dirección MAC del ESP32-S3

```
#include <WiFi.h>

void setup() {
  Serial.begin(115200);

  WiFi.mode(WIFI_STA);
  Serial.println("Direccion MAC en modo estacion:");
  Serial.println(WiFi.macAddress());

  WiFi.mode(WIFI_AP);
```

```
  WiFi.softAP("ESP32_Prueba_MAC", "12345678");

  Serial.println("Direccion MAC del Access Point:");
  Serial.println(WiFi.softAPmacAddress());
}

void loop() {
  // No se requiere codigo en loop para este ejemplo.
}
```

Actividad. Cargue el código, abra el Monitor Serial a 115200 baudios y registre la dirección MAC reportada.

4. Fase 2: capa IP y dirección IP del ESP32-S3

La dirección IP permite identificar un dispositivo dentro de una red. Cuando el ESP32-S3 crea su propio hotspot, normalmente toma la dirección 192.168.4.1. Los celulares o computadores que se conectan a esa red reciben direcciones del mismo segmento, por ejemplo 192.168.4.2 o 192.168.4.3.

Código 2. Crear hotspot y consultar la IP del ESP32-S3

```
#include <WiFi.h>

const char* ssid = "MonitorBioESP32";
const char* password = "bio12345";

void setup() {
  Serial.begin(115200);

  WiFi.mode(WIFI_AP);
  WiFi.softAP(ssid, password);

  IPAddress ip = WiFi.softAPIP();

  Serial.println("Access Point iniciado");
  Serial.print("SSID: ");
  Serial.println(ssid);
  Serial.print("IP del ESP32: ");
  Serial.println(ip);
  Serial.print("MAC del AP: ");
  Serial.println(WiFi.softAPmacAddress());
}

void loop() {
  // El ESP32 mantiene activa la red Wi-Fi.
}
```

Actividad. Después de cargar el código, conecte el celular a la red MonitorBioESP32. Si el celular indica que la red no tiene internet, mantenga la conexión. Esta red sirve para comunicarse localmente con el ESP32.

5. TCP, UDP, puertos y capa de aplicación

TCP y UDP son protocolos de transporte. TCP se usa cuando se desea una comunicación confiable y ordenada, como ocurre al cargar una página web. UDP se usa cuando se prefiere menor latencia y menor sobrecarga, por ejemplo en algunas aplicaciones de video, audio o telemetría simple.

Un puerto permite diferenciar servicios dentro de un mismo dispositivo. En esta práctica, el ESP32-S3 ejecutará un servidor web en el puerto 80, que es el puerto tradicional de HTTP. Cuando el navegador abre http://192.168.4.1, en realidad se conecta al servidor HTTP del ESP32 usando TCP y el puerto 80.

Capa de aplicación. La aplicación será una página web. HTTP es solo uno de muchos protocolos de aplicación. Otros ejemplos son SMTP para correo electrónico, FTP para transferencia de archivos, DNS para resolver nombres de dominio y MQTT para mensajería liviana en IoT.

6. Fase 3: página web estática con lectura de potenciómetro

Primera aplicación web. En este ejemplo el ESP32-S3 crea una red Wi-Fi, levanta un servidor web y muestra en una página el voltaje leído desde un potenciómetro conectado al GPIO4. La página se genera cuando el navegador la solicita. Si el valor cambia, se debe refrescar la página para ver una nueva lectura.

Código 3. Servidor web con página estática y lectura de GPIO4

```
#include <WiFi.h>
#include <WebServer.h>

const char* ssid = "MonitorBioESP32";
const char* password = "bio12345";

const int PIN_POT = 4;
WebServer server(80);

float leerVoltaje() {
  int adc = analogRead(PIN_POT);
  return adc * 3.3 / 4095.0;
}

void handleRoot() {
  float voltaje = leerVoltaje();

  String html = "<!DOCTYPE html><html><head>";
  html += "<meta name='viewport' content='width=device-width, initial-scale=1'>";
  html += "<title>ESP32 Monitor</title>";
  html += "<style>body{font-family:Arial;text-align:center;margin-top:40px;}";
  html += ".card{display:inline-block;padding:20px;border:1px solid #ccc;border-radius:12px;}";
  html += "</style></head><body>";
  html += "<h1>Servidor web ESP32-S3</h1>";
  html += "<div class='card'>";
  html += "<h2>Lectura del potenciometro</h2>";
  html += "<p>Voltaje: <b>" + String(voltaje, 3) + " V</b></p>";
  html += "<p>Recargue la pagina para actualizar.</p>";
  html += "</div></body></html>";

  server.send(200, "text/html", html);
}

void setup() {
  Serial.begin(115200);
  analogReadResolution(12);

  WiFi.mode(WIFI_AP);
  WiFi.softAP(ssid, password);

  Serial.print("Conectese a la red: ");
  Serial.println(ssid);
  Serial.print("Abra en el navegador: http://");
  Serial.println(WiFi.softAPIP());
```

```
  server.on("/", handleRoot);
  server.begin();
  Serial.println("Servidor web iniciado en el puerto 80");
}

void loop() {
  server.handleClient();
}
```

Actividad. Conecte un potenciómetro al GPIO4, cargue el código y abra la IP del ESP32 desde el navegador del celular. Cambie el potenciómetro y actualice manualmente la página para observar el nuevo valor.

7. Fase 4: página dinámica con JavaScript

JavaScript es un lenguaje de scripting que se ejecuta en el navegador. Permite que una página web sea dinámica, es decir, que pueda actualizar información sin recargar toda la página. En este ejemplo, la página solicita cada segundo un dato al ESP32 mediante la ruta /datos. El ESP32 responde con un texto en formato JSON.

Tecnologías web. HTML define la estructura de la página, CSS define la apariencia visual y JavaScript define el comportamiento dinámico. El desarrollo web es un campo amplio de la ingeniería y puede aplicarse tanto a IoT como a sistemas empresariales, salud digital, aplicaciones móviles y plataformas profesionales.

Código 4. Página dinámica con JavaScript y endpoint /datos

```
#include <WiFi.h>
#include <WebServer.h>

const char* ssid = "MonitorBioESP32";
const char* password = "bio12345";

const int PIN_POT = 4;
WebServer server(80);

float leerVoltaje() {
  int adc = analogRead(PIN_POT);
  return adc * 3.3 / 4095.0;
}

void handleDatos() {
  float voltaje = leerVoltaje();
  String json = "{";
  json += "\"voltaje\":" + String(voltaje, 3);
  json += "}";
  server.send(200, "application/json", json);
}

void handleRoot() {
  String html = R"rawliteral(
<!DOCTYPE html>
<html>
<head>
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>ESP32 dinamico</title>
  <style>
    body { font-family: Arial; text-align: center; margin-top: 40px; }
    .card { display: inline-block; padding: 22px; border-radius: 14px;
            box-shadow: 0 0 12px #bbb; min-width: 260px; }
    .valor { font-size: 2.4em; color: #0f4c5c; }
  </style>
</head>
<body>
  <h1>ESP32-S3 - Pagina dinamica</h1>
```

```
  <div class="card">
    <h2>Potenciometro en GPIO4</h2>
    <div class="valor"><span id="voltaje">--</span> V</div>
    <p>Actualizacion automatica cada segundo</p>
  </div>

  <script>
    function actualizarDatos() {
      fetch('/datos')
        .then(response => response.json())
        .then(data => {
          document.getElementById('voltaje').innerText = data.voltaje.toFixed(3);
        });
    }

    setInterval(actualizarDatos, 1000);
    actualizarDatos();
  </script>
</body>
</html>
)rawliteral";

  server.send(200, "text/html", html);
}

void setup() {
  Serial.begin(115200);
  analogReadResolution(12);

  WiFi.mode(WIFI_AP);
  WiFi.softAP(ssid, password);

  Serial.print("Conectese a la red: ");
  Serial.println(ssid);
  Serial.print("Abra en el navegador: http://");
  Serial.println(WiFi.softAPIP());

  server.on("/", handleRoot);
  server.on("/datos", handleDatos);
  server.begin();
}

void loop() {
  server.handleClient();
}
```

Actividad. Cargue el código, abra la página desde el celular y mueva el potenciómetro. Observe que el valor cambia sin recargar manualmente la página.

8. Interfaz hombre-máquina: por qué usar una página web

Interfaz hombre-máquina (HMI). Una HMI es el medio por el cual una persona interactúa con un sistema. En sistemas embebidos podríamos usar botones, teclados, pantallas pequeñas o LEDs. Sin embargo, cada elemento físico aumenta costo, complejidad, consumo de energía y dificultad de uso.

Ventaja de la web. Muchas personas ya saben usar un teléfono móvil o un navegador. Por eso, una interfaz web local puede reducir costos y mejorar la experiencia de usuario. En lugar de agregar una pantalla grande o un teclado al equipo, el microcontrolador puede publicar una página web y el usuario interactúa desde su propio dispositivo.

9. Integración final: monitor biomédico con interfaz web

Objetivo final. El entregable final ya no usará el potenciómetro. Cada grupo debe integrar el MAX30100, el circuito de temperatura y el servidor web. La página debe actualizar dinámicamente frecuencia cardíaca, saturación de oxígeno y temperatura.

Bibliotecas necesarias. Para este código se requieren WiFi, WebServer, Wire, Adafruit GFX, Adafruit SSD1306 y MAX30100lib. Instálenlas antes de la práctica para evitar retrasos por problemas de conexión a internet.

Código 5. Esbozo de integración final: MAX30100, temperatura y servidor web

```
#include <WiFi.h>
#include <WebServer.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include "MAX30100_PulseOximeter.h"

#define I2C_SDA 8
#define I2C_SCL 9

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1
#define OLED_ADDR 0x3C

const char* ssid = "MonitorBioESP32";
const char* password = "bio12345";

const int PIN_TEMP = 4;

WebServer server(80);
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);
PulseOximeter pox;

float bpm = 0.0;
float spo2 = 0.0;
float temperatura = 0.0;
float vtemp = 0.0;

// Reemplacen estos valores por los obtenidos en su calibracion.
float T01 = 25.0;
float V01 = 3.05;
float T02 = 34.5;
float V02 = 2.10;

unsigned long tAnterior = 0;
const unsigned long intervaloMedicion = 500;

float estimarTemperaturaDesdeVoltaje(float V) {
  return T01 + (V - V01) * (T02 - T01) / (V02 - V01);
}

float leerVoltajePromedio() {
  const int N = 50;
  uint32_t suma = 0;

  for (int i = 0; i < N; i++) {
    suma += analogRead(PIN_TEMP);
  }

  float adc = suma / float(N);
  return adc * 3.3 / 4095.0;
}
```

```
void actualizarOLED() {
  char linea[32];

  display.clearDisplay();
  display.setTextColor(SSD1306_WHITE);
  display.setTextSize(1);
  display.setCursor(0, 0);
  display.println("Monitor signos");

  snprintf(linea, sizeof(linea), "BPM: %4.0f", bpm);
  display.setCursor(0, 16);
  display.println(linea);

  snprintf(linea, sizeof(linea), "SpO2: %4.0f %%", spo2);
  display.setCursor(0, 30);
  display.println(linea);

  snprintf(linea, sizeof(linea), "Temp: %4.1f C", temperatura);
  display.setCursor(0, 44);
  display.println(linea);

  display.display();
}

void actualizarSensores() {
  bpm = pox.getHeartRate();
  spo2 = pox.getSpO2();

  vtemp = leerVoltajePromedio();
  temperatura = estimarTemperaturaDesdeVoltaje(vtemp);

  actualizarOLED();

  char linea[96];
  snprintf(linea, sizeof(linea),
           "BPM:%4.0f SpO2:%4.0f Temp:%4.1f Vtemp:%1.3f",
           bpm, spo2, temperatura, vtemp);
  Serial.println(linea);
}

void handleDatos() {
  String json = "{";
  json += "\"bpm\":" + String(bpm, 0) + ",";
  json += "\"spo2\":" + String(spo2, 0) + ",";
  json += "\"temperatura\":" + String(temperatura, 1) + ",";
  json += "\"vtemp\":" + String(vtemp, 3);
  json += "}";

  server.send(200, "application/json", json);
}

void handleRoot() {
  String html = R"rawliteral(
<!DOCTYPE html>
<html>
<head>
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Monitor de signos vitales</title>
  <style>
    body { font-family: Arial; background: #f4f8fb; text-align: center; }
    h1 { color: #0b2545; }
    .card { background: white; display: inline-block; margin: 10px; padding: 18px;
            border-radius: 16px; box-shadow: 0 0 12px #ccd; min-width: 220px; }
    .valor { font-size: 2.2em; color: #0f4c5c; font-weight: bold; }
    .unidad { color: #555; }
  </style>
</head>
```

```
<body>
  <h1>Monitor de signos vitales</h1>

  <div class="card">
    <h2>Frecuencia cardiaca</h2>
    <div class="valor"><span id="bpm">--</span></div>
    <div class="unidad">bpm</div>
  </div>

  <div class="card">
    <h2>Saturacion de oxigeno</h2>
    <div class="valor"><span id="spo2">--</span></div>
    <div class="unidad">%</div>
  </div>

  <div class="card">
    <h2>Temperatura</h2>
    <div class="valor"><span id="temp">--</span></div>
    <div class="unidad">C</div>
  </div>

  <p>Actualizacion automatica cada segundo desde el ESP32-S3</p>

  <script>
    function actualizarDatos() {
      fetch('/datos')
        .then(response => response.json())
        .then(data => {
          document.getElementById('bpm').innerText = data.bpm.toFixed(0);
          document.getElementById('spo2').innerText = data.spo2.toFixed(0);
          document.getElementById('temp').innerText = data.temperatura.toFixed(1);
        });
    }

    setInterval(actualizarDatos, 1000);
    actualizarDatos();
  </script>
</body>
</html>
)rawliteral";

  server.send(200, "text/html", html);
}

void setup() {
  Serial.begin(115200);
  analogReadResolution(12);

  Wire.begin(I2C_SDA, I2C_SCL);

  if (!display.begin(SSD1306_SWITCHCAPVCC, OLED_ADDR)) {
    Serial.println("No se detecta la pantalla OLED");
    while (true);
  }

  display.clearDisplay();
  display.setTextColor(SSD1306_WHITE);
  display.setTextSize(1);
  display.setCursor(0, 0);
  display.println("Inicializando...");
  display.display();

  if (!pox.begin()) {
    Serial.println("No se detecta el MAX30100");
    display.clearDisplay();
    display.setCursor(0, 0);
    display.println("Error MAX30100");
    display.display();
    while (true);
```

```
  }

  pox.setIRLedCurrent(MAX30100_LED_CURR_7_6MA);

  WiFi.mode(WIFI_AP);
  WiFi.softAP(ssid, password);

  Serial.print("Red Wi-Fi creada: ");
  Serial.println(ssid);
  Serial.print("Abra en el navegador: http://");
  Serial.println(WiFi.softAPIP());

  server.on("/", handleRoot);
  server.on("/datos", handleDatos);
  server.begin();

  display.clearDisplay();
  display.setCursor(0, 0);
  display.println("Servidor listo");
  display.println(WiFi.softAPIP());
  display.display();
}

void loop() {
  pox.update();
  server.handleClient();

  if (millis() - tAnterior >= intervaloMedicion) {
    tAnterior = millis();
    actualizarSensores();
  }
}
```

Nota. Este código es un punto de partida. Cada grupo debe reemplazar los valores de calibración T01, V01, T02 y V02 por los obtenidos en su práctica. También debe verificar que el pin PIN_TEMP corresponda al pin ADC usado realmente en su montaje.

10. Entregable de la práctica

- Mostrar al profesor la red Wi-Fi creada por el ESP32-S3.
- Indicar la dirección MAC y la dirección IP del dispositivo.
- Acceder desde un celular o computador a la página web del ESP32.
- Mostrar una página dinámica que actualice frecuencia cardíaca, SpO2 y temperatura.
- Explicar brevemente cómo se relaciona el montaje con las capas: acceso a la red, IP, TCP y aplicación.
- Mantener funcionando la visualización local en OLED si el montaje lo permite.

11. Ir más allá: mejorar la interfaz

El límite es su imaginación. Una vez que el sistema funciona, pueden mejorar la interfaz web con colores, tarjetas visuales, íconos, barras de progreso, alertas o gráficas en tiempo real. También pueden buscar ejemplos de librerías JavaScript para graficar datos en el navegador. Este tipo de desarrollo es muy útil en IoT y también es una ruta profesional interesante: desarrollo web, salud digital, telemetría, dispositivos conectados y aplicaciones biomédicas.

12. Síntesis del proyecto completo

Fase 1. Se implementó el uso del MAX30100 para medir frecuencia cardíaca y saturación de oxígeno, entendiendo el uso de señales ópticas roja e infrarroja y la comunicación I2C.

Fase 2. Se incorporó la medición de temperatura usando un termistor NTC, acondicionamiento analógico, fuente de corriente, amplificación, lectura ADC y calibración de dos puntos.

Fase 3. Se agrega comunicación en red: el ESP32-S3 crea una red Wi-Fi local, ejecuta un servidor web y permite visualizar los signos vitales desde un navegador.

Reflexión final. Este proyecto conecta lo aprendido en circuitos eléctricos, electrónica, sistemas embebidos, bioinstrumentación y redes. Lo desarrollado es el paso inicial hacia un equipo biomédico conectado: sensores, acondicionamiento, procesamiento, visualización y comunicación. Esa integración es muy cercana a los problemas que encontrarán profesionalmente como ingenieros biomédicos.

13. Devolución de componentes al profesor

Una vez el grupo entregue el proyecto funcionando, deberá devolver al profesor los componentes prestados por el laboratorio: el sensor de temperatura tipo NTC, los amplificadores operacionales rail-to-rail (dos: MCP6002) y el sensor MAX30100. Esta devolución debe hacerse en el momento de la entrega del proyecto, verificando que los componentes se encuentren en buen estado y funcionando correctamente.

Esta medida no busca ser restrictiva, sino garantizar que el stock de dispositivos del laboratorio se conserve en buenas condiciones y pueda seguir siendo utilizado por otros grupos en futuras prácticas. En experiencias de laboratorio es común que, si no se hace una devolución controlada, algunos componentes dañados o quemados se mezclen con el material funcional, lo cual genera pérdidas de tiempo, diagnósticos incorrectos y dificultades para las siguientes actividades.

Por esta razón, cada grupo debe ser cuidadoso con el manejo de los componentes prestados. Si alguno de los dispositivos se daña por una mala conexión, sobrevoltaje, polaridad invertida o manipulación inadecuada, el grupo deberá informarlo oportunamente y responder por su reposición con un componente equivalente nuevo.